NLP Exercise 3

Malak Laham and Zenab Waked

<u>Theoretical part:</u>

Question 1

Transition probabilities:

- P(L\H)) =0.5, P(H\H) =1−0.5=0.5
- P(H\L) =0.4, P(L\L) =1−0.4=0.6

Emission probabilities:

- From H:
  P(A\H) =0.2, P(C\H) =0.3, P(G\H) =0.3, P(T\H) =0.2
- From L:
  P(A\L) =0.3, P(C\L) =0.2, P(G\L) =0.2, P(T\L) =0.3

S=ACCGTGCA

According to Viterbi algorithm:

Step 1: First Nucleotide (A)

For H: δ1(H)=P(H\H) ·P(A\H) = 0.5·0.2=0.1

For L: δ1(L)=P(L\H) ·P(A\L) =0.5·0.3=0.15

Maximum: max (0.1,0.15) =0.15

Label: L

Step 2: Second Nucleotide (C)

For H: δ2(H)=max[δ1(H)·P(H\H), δ1(L)·P(H\L)] ·P(C\H)

δ2(H) = max [0.1·0.5, 0.15·0.4] ·0.3=0.018

For L: δ2(L) = max[δ1(H)·P(L\H), δ1(L)·P(L\L)] ·P(C\L)

δ2(L)= max [0.1·0.5, 0.15·0.6] ·0.2 = 0.018

Maximum: Both probabilities are equal at 0.018, ***assume H in tie situations like these.***

Label: H

Step 3: Third Nucleotide (C)

For H: δ3(H)=max [δ2(H)·P(H\H), δ2(L)·P(H\L)] ·P(C\H)

δ3(H)=max [0.018·0.5,0.018·0.4] ·0.3=0.0027

For L: $\delta3(L)=\max[\delta2(H)\cdot P(L\backslash H), \delta2(L)\cdot P(L\backslash L)]\cdot P(C\backslash L)$

$\delta3(L)=\max[0.018\cdot0.5, 0.018\cdot0.6]\cdot0.2=0.00216$

Maximum: max (0.0027,0.00216) =0.0027

Label: H

Step 4: Fourth Nucleotide (G)

For H: $\delta4(H)=\max[\delta3(H)\cdot P(H\backslash H), \delta3(L)\cdot P(H\backslash L)]\cdot P(G\backslash H)$

$\delta4(H)=\max[0.0027\cdot0.5, 0.00216\cdot0.4]\cdot0.3 = 0.000405$

For L: $\delta4(L)=\max[\delta3(H)\cdot P(L\backslash H), \delta3(L)\cdot P(L\backslash L)]\cdot P(G\backslash L)$

$\delta4(L)=\max[0.0027\cdot0.5, 0.00216\cdot0.6]\cdot0.2 = 0.00027$

Maximum: max (0.000405,0.00027)=0.000405

Label: H

Step 5: Fifth Nucleotide (T)

For H: $\delta5(H)=\max[\delta4(H)\cdot P(H\backslash H), \delta4(L)\cdot P(H\backslash L)]\cdot P(T\backslash H)$

$\delta5(H)=\max[0.000405\cdot0.5, 0.00027\cdot0.4]\cdot0.2 = 0.0000405$

For L: $\delta5(L)=\max[\delta4(H)\cdot P(L\backslash H), \delta4(L)\cdot P(L\backslash L)]\cdot P(T\backslash L)$

$\delta5(L)=\max[0.000405\cdot0.5, 0.00027\cdot0.6]\cdot0.3 = 0.00006075$

Maximum: max [(0.0000405,0.00006075)=0.00006075

Label: L

Step 6: Sixth Nucleotide (G)

For H: $\delta6(H)=\max[\delta5(H)\cdot P(H\backslash H), \delta5(L)\cdot P(H\backslash L)]\cdot P(G\backslash H)$

$\delta6(H)=\max[0.0000405\cdot0.5, 0.00006075\cdot0.4]\cdot0.3 = 0.00000729$

For L: $\delta6(L)=\max[\delta5(H)\cdot P(L\backslash H), \delta5(L)\cdot P(L\backslash L)]\cdot P(G\backslash L)$

$\delta6(L)=\max[0.0000405\cdot0.5, 0.00006075\cdot0.6]\cdot0.2 = 0.00000729$

Maximum: Both are equal at 0.00000729 assume H.

Label: H

Step 7: Seventh Nucleotide (C)

For H: $\delta 7(H)=\max [\delta 6(H)\cdot P(H\backslash H), \delta 6(L)\cdot P(H\backslash L)]\cdot P(C\backslash H)$

$\delta 7(H)=\max [0.00000729\cdot 0.5, 0.00000729\cdot 0.4]\cdot 0.3 = 0.0000010935$

For L: $\delta 7(L)=\max [\delta 6(H)\cdot P(L\backslash H), \delta 6(L)\cdot P(L\backslash L)]\cdot P(C\backslash L)$

$\delta 7(L)=\max [0.00000729\cdot 0.5, 0.00000729\cdot 0.6]\cdot 0.2 = 0.0000008748$

Maximum: max [(0.0000010935,0.0000008748)=0.0000010935

Label: H

Step 8: Eighth Nucleotide (A)

For H: $\delta 8(H)=\max [\delta 7(H)\cdot P(H\backslash H), \delta 7(L)\cdot P(H\backslash L)]\cdot P(A\backslash H)$

$\delta 8(H)=\max [0.0000010935\cdot 0.5, 0.0000008748\cdot 0.4]\cdot 0.2 = 0.00000010935$

For L: $\delta 8(L)=\max [\delta 7(H)\cdot P(H\backslash L), \delta 7(L)\cdot P(L\backslash L)]\cdot P(A\backslash L)$

$\delta 8(L)=\max [0.0000010935\cdot 0.5, 0.0000008748\cdot 0.6]\cdot 0.3 = 0.000000164025$

Maximum: max [(0.00000010935,0.000000164025)=0.000000164025

Label: L

Final Sequence of Labels: H→L→H→H→H→L→H→H→L

Question 2

$K_{-2} = K_{-1} = K_0 = \{*\}, K_k = K = \{all\ labels\}\ for\ k = 1 \ldots n.$

$\pi(k, y_1, y_2, y_3)$: $Maximum\ probability\ of\ a\ tag\ sequence\ ending\ with\ y_1, y_2, y_3\ at\ position\ k.$

$bp(k, y_1, y_2, y_3)$: $backpointer\ for\ position\ k$

V: $Vocabulary$

Input: integer n, transition_probabilities, emission_probabilities, V

Initialization

$For\ k = -2, -1, 0$: $\pi(k,*,*,*) = 1$ (start state has probability 1)

$bp(0,*,*,*) = None$

Algorithm

$For\ k = 1..n$

$$\pi(k, y_1, y_2, y_3) = \max_{y_1, y_2, y_3 \in K,\ x_m \in V} [\pi(k-1, y_4, y_1, y_2) \cdot q(y_3 | y_4, y_1, y_2) \cdot e(x_m | y_3)]$$

$$bp\ (k, y_1, y_2, y_3) = \arg\max_{y_1, y_2, y_3 \in K,\ x_m \in V} [\pi(k-1, y_4, y_1, y_2) \cdot q(y_3 | y_4, y_1, y_2) \cdot e(x_m | y_3)]$$

Transition to STOP tag:

$$\pi(n+1, y_1, y_2, STOP) = \max_{y_1, y_2, STOP \in K} [\pi(k-1, y_4, y_1, y_2) \cdot q(STOP | y_4, y_1, y_2)]$$

(the sentence has already ended; we don't want to maximize any emission values).

$$bp(n+1, y_1, y_2, STOP) = \arg\max_{y_1, y_2, STOP \in K} [\pi(k-1, y_4, y_1, y_2) \cdot q(STOP | y_4, y_1, y_2)$$

Backtracking

1. Start from the best word and tag that ends the sentence

   argmax ($bp\ (n+1, :, :, :)$)

2. For k=n, n−1, ...,1:

   Use the back-pointer table $bp\ (k, y_1, y_2, y_3)$ to retrieve:

   - The most probable previous tag t.

        - The most probable word x_m for position k.
    - Append t to the tag sequence and x_m to the word sequence.
3. Reverse the tag and word sequences to get them in the correct order.

Return $(x_1, x_2, \ldots, x_n), (y_1, y_2, \ldots, y_n)$

Practical part:

Part b: evaluate error rates with MLE estimation

Known Words Error Rate: 0.0704

Unknown Words Error Rate: 0.7504

Total Error Rate: 0.1481

Part ciii: Evaluate viterbi without Laplace smoothing or pseudo words

Known Words Error Rate: 0.2968

Unknown Words Error Rate: 0.7365

Total Error Rate: 0.3471

Comparison for Part ciii: Evaluate Viterbi Without Laplace Smoothing or Pseudo-Words

Known Words Error Rate: The bigram HMM tagger has a higher error rate for known words (29.6%) compared to the MLE tagger (7.04%). We assume that this is because the MLE tagger directly assigns the most frequent tag for each word, which works well for known words. Since the bigram HMM considers context, it can introduce small errors when the surrounding tags in the test data differ from the training data.

Unknown Words Error Rate: For unknown words, the bigram HMM tagger performs better (error rate 73.65%) compared to the MLE tagger (error rate 75.04%). The MLE tagger assigns "NN" (noun) to all unknown words as instructed. While this works for some nouns, it fails for verbs, adjectives, and other parts of speech, leading to many errors. In contrast, the bigram HMM tagger uses context (the tags of the previous words) to make predictions for unknown words. Even if the word itself is not in the training set, the surrounding context helps guide the model to a more accurate guess.

Total Error Rate: For the total error rate, the MLE tagger (14.81%) performs better than the bigram HMM tagger (34.71%). We assume that this is because the MLE tagger does a great job at tagging known words, which make up the majority of the test set. The bigram HMM tagger, while better for unknown words, introduces minor errors for known words due to its reliance on context, increasing the overall error rate.

Comparison for Part d: Evaluate Viterbi with Laplace Smoothing and No Pseudo-Words

Part d: Evaluate viterbi with Laplace smoothing and no pseudo words

Known Words Error Rate: 0.3013

Unknown Words Error Rate: 0.7426

Total Error Rate: 0.3517

Known Words Error Rate: Using Add-One smoothing, the known words error rate increases to 30.13%, compared to 7.04% in the MLE tagger and 29.68% in the bigram HMM tagger without smoothing. This likely happens because smoothing adjusts the probabilities to account for unseen events. While this is expected to help, it can reduce the accuracy for high-frequency tags in the training set. Rare events or unseen events are given artificial probabilities, even if they are not relevant to the context. The MLE tagger remains better for known words since it assigns the most frequent tag directly, without altering the probabilities.

Unknown Words Error Rate: The Add-One smoothing error rate is 74.26%, which is slightly better than the MLE tagger (75.04%) but slightly worse than the bigram HMM without smoothing (73.65%). Smoothing helps by giving all tags, even rare ones, a small chance. This improves handling of unknown words but can cause small mistakes compared to the unsmoothed model.

Total Error Rate: The total error rate with Add-One smoothing is 35.17%, which is higher than the MLE tagger (14.81%) and the bigram HMM without smoothing (34.71%). This happens because smoothing changes the probabilities for known words, making them less accurate, while not improving the handling of unknown words enough to make up for it.

Comparison for Part eii: Evaluate Viterbi Without Laplace Smoothing but with Replacing Pseudo-Words

Part e.ii: Evaluate viterbi without Laplace smoothing but with replacing pseudowords

Known Words Error Rate: 0.3325

Unknown Words Error Rate: 0.0000

Total Error Rate: 0.3325

Known Words Error Rate: With pseudo-words, the error rate for known words increases significantly to 33.25%, compared to 7.04% in the MLE tagger, 29.68% in the bigram HMM without smoothing, and 30.13% in the smoothed HMM. We assume that this happens because it now includes the error rate for what was previously defined as unknown words. At the same time, it's logical to think that since a large number of words with low frequency are grouped to a small group of words boosting the pseudo words frequency, they can reduce accuracy for tagging actual frequent known words.

Unknown Words Error Rate: The error rate for unknown words is 0 since there are no unknown words and all have been replaced with a pseudo word (apparently all pseudo words we're also present in the training set after all.)

.

Total Error Rate: The total error rate with pseudo-words is 33.25%, is higher than the MLE tagger (14.81%), yet lower than the the bigram HMM without smoothing (34.71%), and the smoothed HMM (35.17%). This shows us that despite the increase in the known words' error rate, the overall performance of the model is better than that without smoothing or with smoothing. This shows us that in general, grouping infrequent and unknown words into a set of known words according to mutual features and giving these words "emission probabilities" enhances the model's performance.

e.iii)

Most Frequent Errors:

True Tag: ., Predicted Tag: NN, Count: 487

True Tag: NNS, Predicted Tag: NN, Count: 355

True Tag: IN, Predicted Tag: NN, Count: 201

True Tag: JJ, Predicted Tag: NN, Count: 161

True Tag: NP, Predicted Tag: NN, Count: 133

True Tag: VB, Predicted Tag: NN, Count: 123

True Tag: AT, Predicted Tag: NN, Count: 98

True Tag: ,, Predicted Tag: NN, Count: 98

True Tag: RB, Predicted Tag: NN, Count: 78

True Tag: VBN, Predicted Tag: NN, Count: 75

We can see that the most frequent errors occur when predicting "NN". Although our model doesn't use 'NN' for unknown words, and specifically since the model with pseudowords doesn't seem to have any unknown words (each pseudoword replaced a low frequency word in the training data at least once), we can infer the possible bias for 'NN' as follows:

- The "NN" tag naturally dominates both emission and transition probabilities (the tag is very frequent), leading to the model to choose it in the Viterbi path more frequently.
- Dominance of "NN" when mapping infrequent and unknown words to pseudowords. If a high percent of the words mapped to pseudoword 'x' are "NN" tagged words, the emission rate e('x', "NN") will be very high leading to a bias toward it.

Known Words Error Rate: 0.3428

Unknown Words Error Rate: 0.0000

Total Error Rate: 0.3428

Comparison for Part eiii: Evaluate Viterbi with Laplace Smoothing and Replacing with Pseudo-Words

Known Words Error Rate = Total error rate: With pseudo-words and smoothing, the error rate for known words increases to 34.28%, compared to 7.04% in the MLE tagger, 29.68% in the bigram HMM without smoothing, decreases from 35.17% in the smoothed HMM, and increases to 33.25% from the model with pseudo-words alone. We assume that this is because as seen already, Laplacian smoothing disrupts the probabilities for frequent known words, reducing accuracy; whereas applying pseudowords enhances the model (and thus the decrease from 35.17% in the smoothing only model).

Unknown Words Error Rate: Same as previous section (e.ii)

In our example we can see that the regular model without smoothing or pseudo-words performs the best overall, whereas the pseudo words replacing model performs better for unknown words.